

Demonstration of LatencyA

Demonstration of “LatencyA”

The package “LatencyA” is developed for analysis in longitudinal and/or survival studies. The core idea of the package is to use some parametric functions to model the coefficients for a set of correlated variables in a regression. These set of correlated variables are usually exposures measured in different time, i.e. lagged exposures. Because of the high correlation across these variables, the usual regression coefficients can be unstable. Introducing parametric restrictions for these coefficients is a practical solution.

Installation

The package is still on github. To install it, please run the code below,

```
library(devtools)
```

Loading required package: usethis

```
install_github("fyGuo/LatencyA")
```

Skipping install of 'LatencyA' from a github remote, the SHA1 (a378eeff) has not changed since last install. Use `force = TRUE` to force installation

```
library(LatencyA)
```

Warning: replacing previous import 'Hmisc::src' by 'dplyr::src' when loading 'LatencyA'

Warning: replacing previous import 'Hmisc::summarize' by 'dplyr::summarize' when loading 'LatencyA'

```
method          from
all.equal.connection parallelly
```

```
Warning: replacing previous import 'dplyr::filter' by 'stats::filter' when
loading 'LatencyA'
```

The demo data

```
## sim_data is an example data for the package
head(sim_data)
```

2

314	-59.15481	54.94394	-61.54595	144.93788	-85.74988	-79.885532	124.78121
	lag18	lag19	lag20	lag21	lag22	lag23	lag24
31	173.21311	131.16251	33.000463	271.50636	-2.053207	55.09861	-68.05664
35	-26.42162	-75.76387	3.082798	113.54420	173.213107	131.16251	33.00046
311	-86.20440	-53.50364	-67.695038	-180.78211	-40.232013	122.16364	31.41330
312	-19.05053	72.08505	23.782001	-57.70099	77.777960	-165.64449	-62.80647
313	102.61266	-86.95447	-46.796228	63.65471	-67.877028	-19.05844	-166.92618
314	-28.51466	-145.76685	3.731421	45.16366	-16.969562	221.66655	235.87058
	lag25	lag26	lag27	lag28	lag29	lag30	id
31	-60.01338	-22.811106	64.77926	132.78760	46.97388	-127.67377	1
35	271.50636	-2.053207	55.09861	-68.05664	-60.01338	-22.81111	1
311	-58.08986	-19.391188	-62.79373	57.91526	-141.07211	68.95981	2
312	110.54688	66.450394	-56.45086	-44.60125	108.97088	-41.06659	3
313	-157.48417	-45.659207	87.89800	-65.45968	33.92690	-92.92682	4
314	-57.36937	6.686643	-117.28430	66.09278	-59.51039	31.55977	5
	cumExposure	lambda	failure_time	failure	age_end		
31	-32.189500	0.01999852	35.561677761	0	51.00000		
35	10.019454	1.36178750	0.406887662	1	54.40689		
311	42.870955	36.37742403	0.012038596	1	51.01204		
312	70.411894	571.37296902	0.001167099	1	50.00117		
313	-1.871566	0.41465694	0.720067399	1	52.72007		
314	-8.869035	0.20596466	1.856737388	1	53.85674		

The data is organized in a “long” format, that is usually required for a time-varying Cox regression and/or mixed-effect models or gee. Each row represents a person-time data, it contains,

- id: the person id
- age_start: the starting age of the time interval
- age_end: the ending age of the time interval
- lag0 to lag30: the lagged exposures from lag0 (current exposure) to lag30 (exposure 30 periods ago).
- failure: the event indicator (1 for event, 0 for censored)

I did not show the other covariates in the data, but they can be easily added to the data frame. I will do such an example in the following analysis.

Core functions

There are two core functions in the package, namely, - CoxNCSpline: use a natural cubic spline to model the coefficients for the exposures. - CoxPoly: use a set of orthonormal polynomials to model the coefficients for the exposures.

I recommend using CoxNCSpline, see the following example

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
knots_number <- 3
latency <- 20
# fit the model
# it requires data, names of the start time, end time, status, exposure variables, number of
fit <- CoxNCSpline(sim_data, time_start, time_end, status, exposure, knots_number, latency)

# extract the estimated log hazard ratio and its variance at lag = 15
extract_CoxNCSpline(fit, lag = 15, latency = 20)
```

```
      log_HR    log_HR_var
1 0.0009725208 4.002855e-09
```

Usually whenever fitting natural cubic splines, the hyper-parameter `knots_number` is required, and has to be somehow tuned. Thus, instead of giving a pre-specified number, the following function allows the researcher to give a vector of possible knot numbers. The function will help choose the optimal one. We will use the function `CoxKnotSearch`, which conducts cross-validation to choose the optimal knot number.

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)

# the possible knot number is 3,4, and 5
knots_number <- c(3,4,5)
latency <- 20

# fit the model and choose the optimal knot number
fit <- CoxKnotsearch(sim_data, time_start, time_end, status, exposure, knots_number, latency)
extract_CoxKnotsearch(fit, lag = 15, latency = 20)
```

```
      [,1]
[1,] 0.001192148
```

```
# to account the variance in the ccross validation we recommend doing bootraps and using the
# which fits and gives the result at the specified lag in one function
result <- CoxKnotsearch_boot(sim_data, time_start, time_end, status, exposure, latency = 20,
```

```
Warning in supportsMulticoreAndRStudio(...): [ONE-TIME WARNING] Forked
processing ('multicore') is not supported when running R from RStudio because
it is considered unstable. For more details, how to control forked processing
or not, and how to silence this warning in future R sessions, see
?parallelly::supportsMulticore
```

```
Warning: package 'future' was built under R version 4.3.3
```

```
Warning: package 'purrr' was built under R version 4.3.3
```

Adjusting for other covariates

The package allows the researcher to adjust for other covariates by choosing the variable name and the adjust functions at they like. For example, I will use X to denote the vector for the exposures, and L to denote the vector for other covariates. The model fitted by the package can be roughly understood as

$$\lambda_t(X, Z) = g(X; \theta) + h(L; \psi)$$

where $g(X; \theta)$ is the fitted function for the exposures, and $h(L; \psi)$ is the function for other covariates. For example, in the following code, $h(L; \psi)$ is $\psi_1 L + \psi_2 L^2$

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
knots <- 0:19
latency <- 20
fit <- CoxTermsearch(sim_data, time_start, time_end, status, exposure, knots = knots, latency = latency)
extract_CoxTermsearch(fit, 10, latency = latency, knot = knots)
```

```
      [,1]
[1,] 0.004512657
```

```
#In this example, we will generate a random covariate L and adjust for it in our model
sim_data$L <- rbinom(nrow(sim_data), 1, 0.5)
adjusted_variable <- "L"
adjusted_model <- "L+I(L^2)"
result <- CoxKnotsearch_boot(sim_data, time_start, time_end, status, exposure, latency = 20,
```